Preentrega de Techlab

Se diseñó una aplicación Java enfocada en el registro y la gestión de productos. Usando así conceptos vistos en estas ocho clases del curso de TechLab; variables, operadores, colecciones (listas), POO (clases, objetos, encapsulamiento, herencia, polimorfismo), excepciones y organización de código en paquetes/módulos. Se fueron diseñando distintos paquetes para separar las tareas de la aplicación y facilitar la escalabilidad del proyecto para una futura migración a una API REST con Spring Boot.

La arquitectura de la aplicación se divide en:

- **Punto de entrada (main)** Main.java
- **Capa de dominio (model)** Producto.java
- **Capa de lógica de negocio (service)** ProductoService.java
- **Capa de interfaz de usuario (ui)** MenuProducto.java
- **Capa transversal (util y exception)** – Validador.java – ProductoNoEncontradoException.java – StockInsuficienteException.java

Descripción de las clases:

Main.java

Clase principal que es el punto de entrada del programa. Se encarga de orquestar la ejecución del mismo y coordinar todas las dependencias.

- Se crea el servicio y el menú
- Se controla el bucle de menú hasta que el usuario decida salir
- Atrapa las excepciones que puedan surgir al mostrar el menú con el try/catch y muestra el mensaje al usuario sin que el programa se cierre.
- Carga de datos de prueba para evitar hacerlo manualmente cada vez que se ejecute el programa.

Producto.java

Se creó una clase producto con atributos privados para aplicar encapsulamiento y evitar su modificación desde afuera de la clase, pero con sus getters y setters para gestionar y regular el acceso/modificación de los atributos.

Luego un constructor de producto con parámetros (pero sin parámetro id para que el usuario no lo modifique, se asigna el id desde ProductoService)

Se definen getters y setters para acceder a cada atributo de Producto por fuera de la clase (id, nombre, precio, stock y categoría).

Se reescribe el método toString() hererado de Object. Se utiliza @Override para indicar que estoy reescribiendo el método toString() en caso de que falle al momento de compilar el programa.

ProductoService.java

Es la capa de servicio que contiene la lógica de negocio del sistema. Se encarga de a colección(persistencia) de productos, asigna el id al guardar un nuevo producto, valida los datos antes de guardar o actualizar y busca, modifica y elimina productos por el id.

No interactúa con el usuario, ya que lo hacemos en el Main. Esta separación después permite cambiar este archivo por un API REST.

Se crea una interfaz List que guarda los productos solo durante la ejecución del programa. Contador para asignar ids únicos

Operaciones del CRUD(Create, Read, Update, Delete):

- Create: agrega un producto nuevo al catálogo, pero se valida antes de guardar. Si pasa las validaciones se le asigna un id, y al encapsularlo se logra que el usuario no lo modifique. Se usa el contador y el método add de los ArrayList.
- Read: devuelve toda l a lista de productos. También se puede obtener el producto por id en caso de que no exista se lanza una excepción.
- Update: actualiza los datos de un producto existente (solo los datos editables, no el id). En el caso de que no exista el id se reutiliza el método de obtenerPorId. Se validan los nuevos datos con los métodos de validación y se modifican.
- Delete: elimina un método por el id. Primero se verifica que exista con el método obtenerPorId y luego se elimina.

MenuProducto.java

Maneja la interacción con el usuario a través del menú de consola. Se encarga de:

- Mostrar el menú al usuario.
- Pedirle los datos necesarios para cada operación
- Mostrar los resultados o mensajes informativos
- Atributos: Scanner y Service que recibe por constructor, para que quien instancie la clase tenga el control sobre qué Scanner y qué Service usar (“Inyección por constructor”).
- Se designan los métodos del menú (agregarProducto, listarProducto, buscarProducto, actualizarProducto y eliminarProducto), utilizando métodos del ProductoService y se validan los mismos

Validador.java

Es una clase utilitaria que agrupa métodos estáticos y constantes de uso común, para usarlos no hay necesidad de instanciar un objeto al ser static. Se invoca directamente con Validador.validarNombre(..).

Las validaciones de los datos del producto (nombre, precio, stock, categoría) lanzan una excepción si el dato es inválido. En el caso de que el dato sea válido, no retornan nada.

Métodos para gestionar los errores por consola ingresados por el usuario, como asegurarse que ingrese un entero válido, decimales y leer un texto.

ProductoNoEncontradoException.java

Excepción personalizada para manejo de error de dominio, que se lanza al buscar un producto por id y el mismo no existe en el sistema. Otorga claridad en la comunicación de errores.

Hereda de Super Clase RuntimeException

StockInsuficienteException.java

Excepción personalizada para manejo de error de dominio, que se lanza al asignar un valor inválido de stock. Otorga claridad en la comunicación de errores.

Hereda de Super Clase RuntimeException